

# Analiza Głównych Składowych

## Principal Component Analysis (PCA)

Paweł Gliwny

Uniwersytet Łódzki  
Wydział Fizyki i Informatyki Stosowanej

Obróbka Danych

# Problem wyjściowy

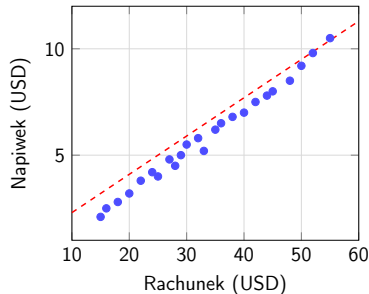
## Obserwacja:

- Zmienne często **współzienne** (kowariancja)
- Część zmienności jednej zmiennej jest powielana przez inną
- Informacja jest **redundantna**

## Przykład:

Rachunek w restauracji  $\leftrightarrow$  Napiwek

Gdy rachunek rośnie, napiwek też rośnie!



## Korelacja dodatnia:

- Wzrost  $\leftrightarrow$  Waga osoby
- Powierzchnia mieszkania  $\leftrightarrow$  Cena
- Temperatura  $\leftrightarrow$  Sprzedaż lodów
- Wykształcenie  $\leftrightarrow$  Zarobki

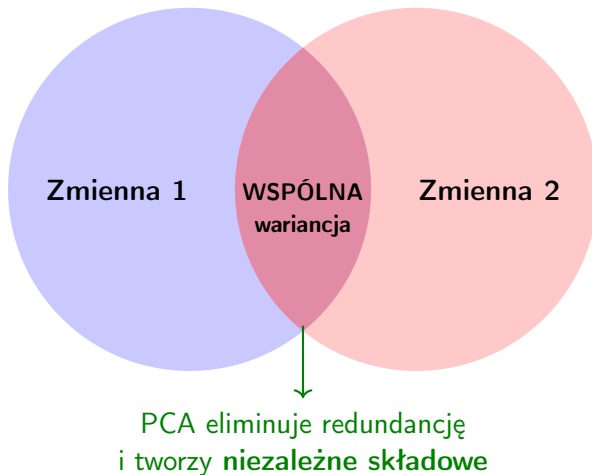
## Korelacja ujemna:

- Przebieg auta  $\leftrightarrow$  Cena auta
- Temperatura  $\leftrightarrow$  Sprzedaż kurtek
- Wiek sprzętu  $\leftrightarrow$  Wydajność

## Wniosek

Skoro zmienne „mówią to samo”  $\rightarrow$  można je skompresować!

# Idea redundancji informacji



# Czym jest PCA?

## Definicja

**PCA** (Principal Component Analysis) – technika redukcji wymiarowości, która przekształca zbiór skorelowanych zmiennych w zbiór **nieskorelowanych** zmiennych zwanych **głównymi składowymi**.

## Kluczowe cechy:

- Łączy wiele zmiennych w mniejszy zestaw **głównych składowych**
- Główne składowe = **ważone kombinacje liniowe** oryginalnych zmiennych
- Zachowuje jak największą część **wariancji** danych
- Działa **tylko dla zmiennych numerycznych**

## Cel

Redukcja wymiarowości przy minimalnej utracie informacji

## Prosty przykład: 2 zmienne

Dane wejściowe: dwie zmienne  $X_1$  i  $X_2$

Dwie główne składowe:

$$Z_1 = w_{1,1} \cdot X_1 + w_{1,2} \cdot X_2 \quad (1)$$

$$Z_2 = w_{2,1} \cdot X_1 + w_{2,2} \cdot X_2 \quad (2)$$

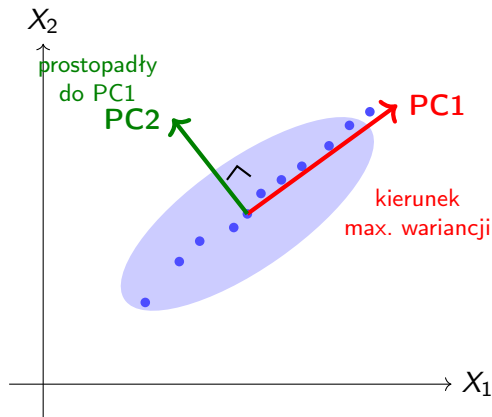
Ładunki (loadings):

- Wagi  $w_{i,j}$  = ładunki składowych
- Określają wkład zmiennych w składowe

Własności:

- $Z_1$  wyjaśnia **maksimum** wariancji
- $Z_2$  jest **ortogonalna** do  $Z_1$
- $Z_2$  wyjaśnia resztę wariancji

# Geometryczna interpretacja PCA



- **PC1** – kierunek największego „rozrzutu” danych
- **PC2** – kierunek prostopadły (ortogonalny)

Dane wejściowe:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p}$$

gdzie:

- $n$  – liczba próbek (wiersze)
- $p$  – liczba cech/zmiennych (kolumny)

Centrowanie danych (odejmowanie średniej):

$$\bar{X} = X - 1_n \mu^T$$

gdzie  $\mu = [\mu_1, \mu_2, \dots, \mu_p]^T$  – wektor średnich



## Definicja

Macierz kowariancji opisuje współzmiennność między wszystkimi parami zmiennych:

$$C = \frac{1}{n-1} \bar{X}^T \bar{X} \in \mathbb{R}^{p \times p}$$

$$C = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_p) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots & \text{Cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(X_p, X_1) & \text{Cov}(X_p, X_2) & \cdots & \text{Var}(X_p) \end{bmatrix}$$

- Macierz **symetryczna**:  $C = C^T$
- Na przekątnej: **wariancje** poszczególnych zmiennych
- Poza przekątną: **kowariancje** między zmiennymi

# Rozkład na wartości i wektory własne (Eigendecomposition)

## Równanie własne

$$Cv_i = \lambda_i v_i$$

gdzie:

- $\lambda_i$  – **wartość własna** (eigenvalue)
  - reprezentuje wariancję wzdłuż  $i$ -tej składowej
  - $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$
- $v_i$  – **wektor własny** (eigenvector)
  - reprezentuje kierunek  $i$ -tej głównej składowej
  - wektory są **ortonormalne**:  $v_i^T v_j = \delta_{ij}$

## Kluczowa obserwacja

Wektory własne macierzy kowariancji = kierunki głównych składowych!

# Macierz transformacji i projekcja

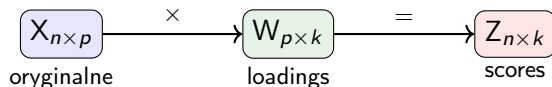
Macierz transformacji (loadings):

$$W = [v_1 \ v_2 \ \cdots \ v_k] \in \mathbb{R}^{p \times k}$$

gdzie  $k \leq p$  to liczba wybranych składowych.

Projekcja danych (scores):

$$Z = \bar{X}W \in \mathbb{R}^{n \times k}$$



## Wyjaśniona wariancja

Procent wyjaśnionej wariancji przez  $i$ -tą składową:

$$\text{Var}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j} \times 100\%$$

Skumulowana wariancja (pierwsze  $k$  składowych):

$$\text{Var}_{1:k} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^p \lambda_j} \times 100\%$$

### Przykład (zbiór IRIS)

|                 | PC1   | PC2   | PC3   | PC4  |
|-----------------|-------|-------|-------|------|
| Wariancja (%)   | 72.96 | 22.85 | 3.67  | 0.52 |
| Skumulowana (%) | 72.96 | 95.81 | 99.48 | 100  |

⇒ PC1 + PC2 wyjaśniają **95.8%** wariancji!

# Dlaczego standaryzacja jest kluczowa?

## Problem

PCA jest wrażliwe na **skale** zmiennych!  
Zmienne o większych wartościach dominują analizę.

Przykład (zbiór Wine):

| Cecha   | Zakres wartości | Wariancja |
|---------|-----------------|-----------|
| Alkohol | 11.0 – 14.8     | 0.66      |
| Prolina | 278 – 1680      | 98609     |

Bez standaryzacji: **Prolina** zdominuje całą analizę!

# Standaryzacja (z-score)

## Wzór standaryzacji

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

gdzie:

- $\mu_j$  – średnia  $j$ -tej cechy
- $\sigma_j$  – odchylenie standardowe  $j$ -tej cechy

**Po standaryzacji:**

- Każda cecha ma  $\mu = 0$  i  $\sigma = 1$
- Wszystkie cechy mają **równy wpływ** na PCA
- Macierz kowariancji  $C$  = macierz korelacji  $R$

## W Pythonie

```
from sklearn.preprocessing import StandardScaler  
X_std = StandardScaler().fit_transform(X)
```

# Algorytm PCA – krok po kroku

- 1 Standaryzacja danych (opcjonalnie, ale zalecane)

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

- 2 Obliczenie macierzy kowariancji

$$C = \frac{1}{n-1} \bar{X}^T \bar{X}$$

- 3 Rozkład na wartości i wektory własne macierzy kowariancji

$$Cv_i = \lambda_i v_i$$

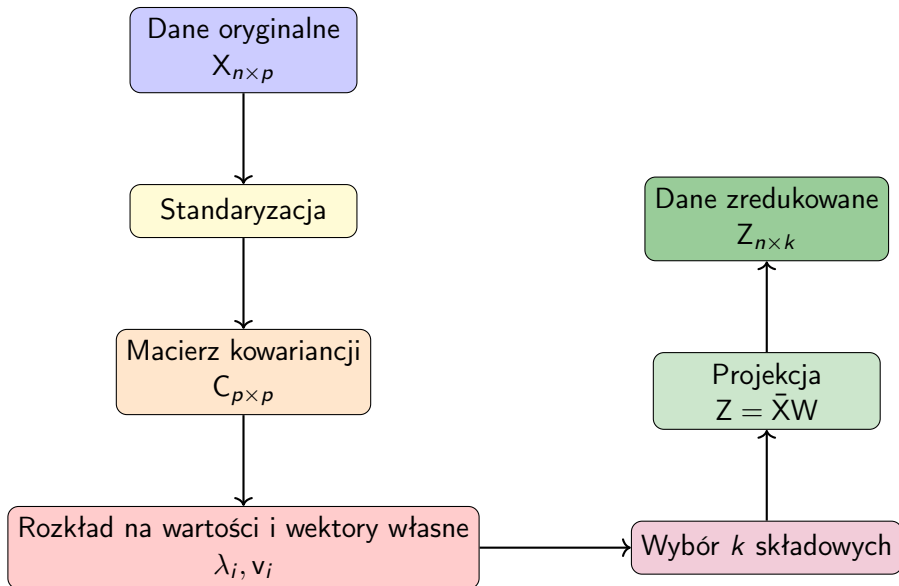
- 4 Sortowanie wektorów własnych wg malejących wartości własnych

- 5 Wybór  $k$  pierwszych składowych

- 6 Projekcja danych na nową przestrzeń

$$Z = \bar{X}W$$

# Schemat przepływu PCA





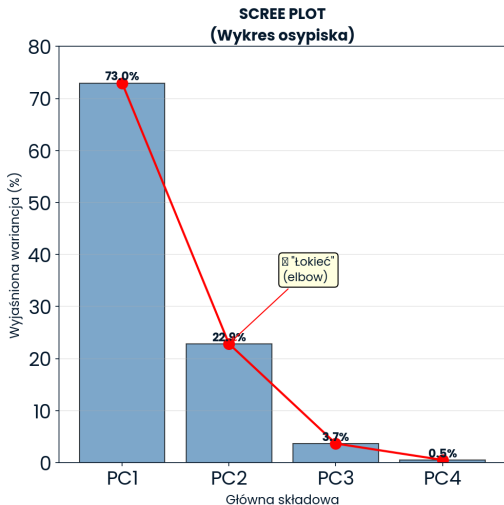
## ① Próg wyjaśnionej wariancji

- Wybieramy  $k$  takie, że  $\text{Var}_{1:k} \geq 80\%$  lub  $95\%$
- Obiektywne kryterium

## ② Metoda łokcia (Scree Plot)

- Szukamy punktu załamania na wykresie
- Subiektywna, ale intuicyjna

# Scree Plot (wykres osypiska)



## Jak czytać?

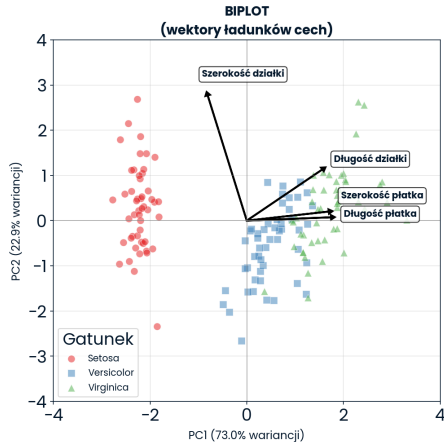
- Oś Y: wartość własna lub % wariancji
- Oś X: numer składowej
- Szukamy łokcia – miejsca załamania

## Reguła:

- Zatrzymujemy składowe **przed** łokciem
- Reszta to шум

Nazwa: scree = osypisko skalne

# Biplot – interpretacja ładunków



## Elementy biplot:

- Punkty – próbki w przestrzeni PC
- Strzałki – wektory ładunków cech

## Interpretacja strzałek:

- Długość – siła wpływu cechy
- Kierunek – korelacja z PC
- Kąt między strzałkami – korelacja między cechami

Strzałki w tym samym kierunku  $\Rightarrow$  cechy skorelowane

| Operacja                             | Wzór                                              |
|--------------------------------------|---------------------------------------------------|
| Standaryzacja                        | $z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$        |
| Macierz kowariancji                  | $C = \frac{1}{n-1} \bar{X}^T \bar{X}$             |
| Rozkład na wartości i wektory własne | $Cv_i = \lambda_i v_i$                            |
| Projekcja (scores)                   | $Z = \bar{X}W$                                    |
| Wyjaśniona wariancja                 | $\frac{\lambda_i}{\sum_j \lambda_j} \times 100\%$ |

# Kiedy stosować PCA?

## Zalety:

- Redukcja wymiarowości
- Usunięcie korelacji między zmiennymi
- Wizualizacja danych wielowymiarowych
- Przyspieszenie uczenia modeli ML
- Redukcja szumu

## Ograniczenia:

- Tylko zmienne numeryczne
- Zakłada liniowe zależności
- Wrażliwość na outliers
- Utrata interpretowalności
- Wymaga standaryzacji

## Typowe zastosowania

- Eksploracyjna analiza danych (EDA)
- Preprocessing dla ML (redukcja wymiarów)
- Kompresja obrazów
- Analiza danych genetycznych, finansowych, sensorowych

## Minimalny przykład

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Standaryzacja
X_std = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_std)

# Wyniki
print(pca.explained_variance_ratio_) # % wariancji
print(pca.components_) # ładunki (loadings)
```

# Dziękuję za uwagę!

Pytania?

**Materiały:**

Przykład + zadania dostępne na Teams